

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF CSE

PROJECT ABSTRACT SUBMISSION

COURSE: COMPILER DESIGN

A.Y. 2026 - 2027

TITLE OF THE PROJECT	An Interactive Context-Free Grammar Analyzer for Parse Tree Generation, Ambiguity Detection, and Grammar Transformation
-----------------------------	--

S. No.	University ID	Name
1	2420090092	Bindu Potukuchi
2	2420030367	Sarvani Topella
3	2420030507	Tanvi Bandaru
4	2420030536	Varshitha Ravuri
Name of the Guide		Dr.Krishna Kishore

1. Abstract

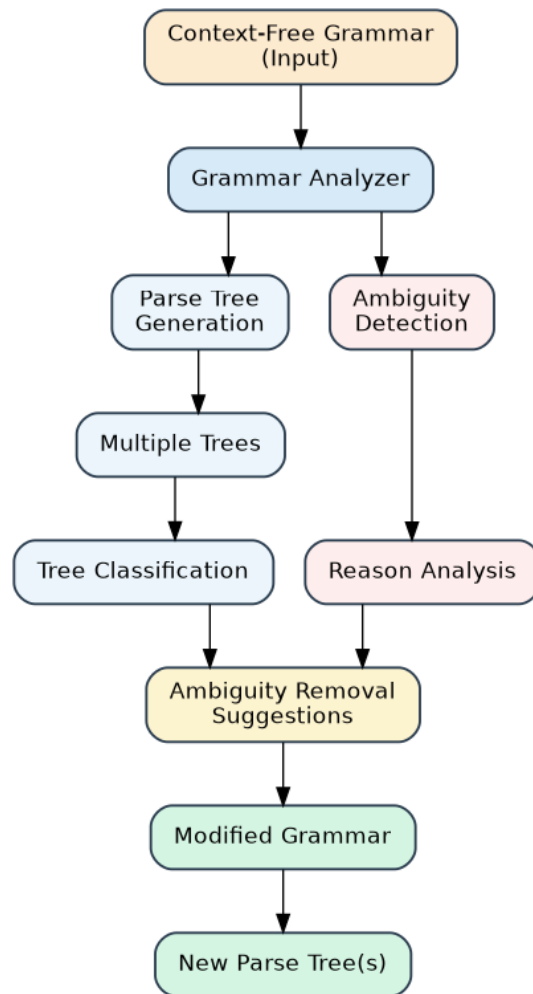
Context-free grammars (CFGs) form the theoretical backbone of compiler design and language processing, yet learners frequently struggle to understand how a grammar generates parse trees and, more importantly, why a grammar may be ambiguous. Existing classroom tools typically produce a single static parse tree for a given input and offer no insight into ambiguity, its underlying cause, or how the grammar itself could be rewritten to eliminate it. This gap forces students to reason about grammar transformations such as left factoring, left-recursion elimination, and precedence assignment purely on paper, without any interactive feedback.

This project proposes an interactive Context-Free Grammar Analyzer that allows a user to input a CFG and a test string, validates the grammar's structure, and generates the corresponding parse tree. When an input string admits more than one valid derivation, the system generates and displays all possible parse trees, flags the grammar as ambiguous, and classifies each derivation (leftmost, rightmost, parse tree, or abstract syntax tree where applicable). The analyzer then explains the source of the ambiguity by highlighting where the derivations diverge, and suggests concrete transformations — left factoring, left-recursion elimination, or the introduction of precedence and associativity rules — to resolve it. Finally, it presents a before-and-after comparison, re-parsing the input against the transformed grammar to visually demonstrate that the ambiguity has been removed.

By turning an abstract theoretical exercise into a visual, step-by-step experience, the tool helps students and developers build intuition for grammar design, ambiguity resolution, and parser behaviour, while remaining extensible to additional compiler-design features such as LL/LR parse table construction and code generation.

2. System Architecture

The diagram below summarizes the end-to-end flow of the Grammar Analyzer, from grammar input through parse tree generation and ambiguity detection, to the suggestion of transformations and generation of a revised, unambiguous grammar.

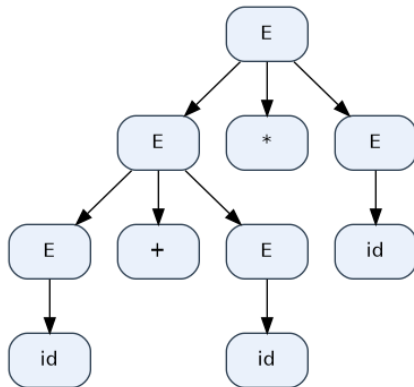


3. Illustrative Example

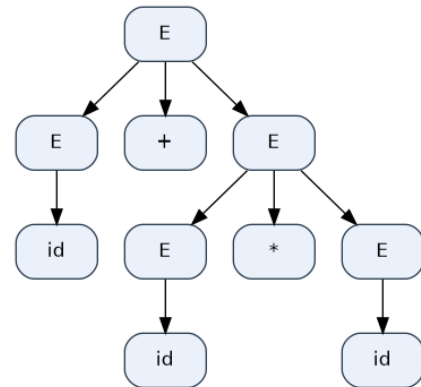
Grammar: $E \rightarrow E + E \mid E * E \mid id$

Input string: $id + id * id$

The analyzer finds two valid parse trees for this input, so the grammar is flagged as ambiguous:



Tree 1: $(id + id) * id$



Tree 2: $id + (id * id)$

⚠ Ambiguous Grammar

The input string has multiple valid parse trees.

Reason: the grammar does not specify precedence between '+' and '*'.

Suggested solution: define '*' with higher precedence than '+'.

Revised grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow id$

Re-parsing the same input against the revised grammar now yields a single, unambiguous parse tree:

